

Egyptian Currency Detection — Model Description

1. Overview

A YOLOv8-nano object-detection model fine-tuned to recognize Egyptian banknotes in images, video, or a live camera feed. For each frame it returns the bounding box, class, and confidence of every detected note, then sums the denominations to produce a running total in Egyptian pounds (EGP).

- **Architecture:** YOLOv8n (Ultralytics)
- **Task:** multi-class object detection (7 classes)
- **Framework:** PyTorch (training) → TorchScript / PyTorch Lite (mobile)
- **Training run:** `outputs/runs/train2/` — 22 epochs, image size 416, batch 4, optimizer auto, cosine LR, seed 42
- **Dataset spec:** `data_yolo/data.yaml`

2. Classes

The class IDs in the dataset and the EGP value used for the running total:

ID	Class	Value (EGP)
0	1_EGP	1
1	5_EGP	5
2	10_EGP	10
3	20_EGP	20
4	50_EGP	50
5	100_EGP	100
6	200_EGP	200

3. Trained artifacts

File	Format	Use
outputs/runs/train2/weights/best.pt	PyTorch	Python inference (server / desktop)
outputs/runs/train2/weights/last.pt	PyTorch	Last training checkpoint
outputs/runs/train2/weights/best.torchscript	TorchScript	Cross-platform deployment
outputs/runs/train2/weights/best.ptl	PyTorch Lite	Mobile (Flutter / Android / iOS)
outputs/best_model.pth	PyTorch state	Backup of best checkpoint

4. Input

- **Type:** RGB image (single frame).
- **Tensor shape expected by the network:** $1 \times 3 \times 416 \times 416$, float, normalized to $[0, 1]$.
- **Source:** any aspect ratio — the inference pipeline letterboxes the frame to 416×416 before feeding the network.
- **Color order:** Python pipeline uses OpenCV (BGR) and converts internally; on Flutter you must pass **RGB**.

5. Output

The Python wrapper (`src/detect.py`) decodes the raw network outputs into a list of detections and emits one JSON file per image with this schema:

```
{
  "detections": [
    {
      "class": "100_EGP",
      "confidence": 0.84,
      "bbox": [x1, y1, x2, y2]
    }
  ],
  "counts": { "100_EGP": 2 },
  "total": 200
}
```

- **bbox** — pixel coords in the original image (top-left, bottom-right).
- **confidence** — float in $[0, 1]$; default keep-threshold is 0.35.
- **counts** — number of detections per class after NMS ($\text{IoU} = 0.5$).
- **total** — sum of denominations across kept detections, in EGP.

Inference also writes an annotated image alongside the JSON, e.g. `outputs/detections/100__100.100_annotated.jpg`.

6. Linking the model with Flutter

The mobile-ready file is `best.ptl` (PyTorch Lite). Flutter consumes it via the `pytorch_lite` plugin. High-level flow:

```
Camera frame ■■ resize 416×416 + normalize ■■ pytorch_lite.runModel
               ■■ list of Detection(class, score, bbox)
               ■■ map class → denomination → sum → display total
```

6.1 Add the plugin

`pubspec.yaml`:

```
dependencies:
  pytorch_lite: ^4.3.2
  camera: ^0.10.5
  image: ^4.1.7
```

6.2 Bundle the model + label file

- Copy `outputs/runs/train2/weights/best.ptl` into

`flutter_app/assets/models/best.ptl`.

- Create `flutter_app/assets/models/labels.txt`:

```
1_EGP
5_EGP
10_EGP
20_EGP
50_EGP
100_EGP
200_EGP
```

- Register both in `pubspec.yaml`:

```
flutter:
  assets:
    - assets/models/best.ptl
    - assets/models/labels.txt
```

6.3 Load and run

```
import 'dart:typed_data';
import 'package:pytorch_lite/pytorch_lite.dart';

class CurrencyDetector {
  static const _labels = [
    '1_EGP', '5_EGP', '10_EGP', '20_EGP',
    '50_EGP', '100_EGP', '200_EGP',
  ];
  static const _values = {
    '1_EGP': 1, '5_EGP': 5, '10_EGP': 10, '20_EGP': 20,
    '50_EGP': 50, '100_EGP': 100, '200_EGP': 200,
  };

  late ModelObjectDetection _model;

  Future<void> load() async {
    _model = await PytorchLite.loadObjectDetectionModel(
      'assets/models/best.ptl',
      _labels.length, // numberOfClasses
      416, 416, // imageWidth, imageHeight
      labelPath: 'assets/models/labels.txt',
      objectDetectionModelType: ObjectDetectionModelType.yolov8,
    );
  }

  Future<({List<ResultObjectDetection> dets, int total})> detect(
```

```

    Uint8List jpegBytes,
  ) async {
    final dets = await _model.getImagePrediction(
      jpegBytes,
      minimumScore: 0.35,
      iOUThreshold: 0.5,
    );
    final total = dets.fold<int>(
      0,
      (sum, d) => sum + (_values[d.className ?? ''] ?? 0),
    );
    return (dets: dets, total: total);
  }
}

```

6.4 Display

`ResultObjectDetection` exposes `className`, `score`, and a normalized `Rect` (`rect.left/top/width/height` in `[0, 1]`). Multiply by the preview widget's size to draw boxes; show `total` in a banner.

6.5 Alternative: keep inference on a server

If on-device inference is too heavy on low-end phones, expose a small FastAPI endpoint that wraps `src/detect.py` and have Flutter POST a JPEG, then render the JSON from section 5. Same contract — only the runtime moves.

7. Performance notes

- Trained on CPU at `imgsz=416` to fit a laptop budget; `.ptl` runs in ~150–400 ms per frame on a mid-range phone. Drop `imgsz` to 320 for ~2× speedup.
- Mosaic was disabled near the end of training (`close_mosaic=10`) to stabilize box regression — banknotes are usually photographed individually rather than in cluttered scenes.
- Confusion-matrix and PR curves are in `outputs/runs/train2/`.

8. Retraining / re-export

```

python src/train.py          # uses src/config.py defaults
python src/evaluate.py       # mAP on the val split
python src/export_ptl.py     # regenerate best.ptl for Flutter
python src/detect.py --image path/to/frame.jpg

```